```
permit(
    principal,
    action == Action::"finance:view_costs",
    resource in ACME::CostCenter
)
when {
    principal.role == "manager" &&
    resource.department_id == principal.department_id
};
```

With that policy in place, a typical request from the Manager Dashboard might look like this:

- Principal: Carol (employee ID, role = manager)
- Action: `finance:view_costs`
- Resource: CostCenter(“MKT-001”)
- Context: calling service = Manager Dashboard

Cedar returns `allow` when Carol requests data for her own cost center, and the proxy forwards the call to the Finance API. If she attempts to view another department’s cost center, Cedar returns `deny`, and the proxy blocks the request before it reaches the service.

The proxy does more than just forward calls. As the policy enforcement point, it is responsible for creating a valid Cedar authorization request by combining the API details (which specify the principal, action, and resource) with identity information from the TLS connection. This way, the Finance API never has to generate PARC requests itself. The proxy ensures every call is normalized and checked against the correct policy before reaching the service.

This pattern allowed ACME to extend Cedar-based authorization to services outside the gateway or mesh. It also created a smaller blast radius: if the Finance proxy failed or was misconfigured, only the Finance API was affected, not the whole system. The trade-off was increased operational overhead from managing and maintaining more proxies. However, for teams not yet migrated to the mesh, per-service proxies ensured Cedar’s policies continued to protect their APIs.